02-lesson

Tags	
Course	🦠 Malwer Analysis
Confidence	Not Confident
Last Edited	3 November 2026 14:39

Okay, guys, can you hear me on back?
With Mike? Okay.
Welcome to the second lecture of the... well, actually, third lecture of this course on.
Malware analysis and incident forensics, I think,
Professor D'Delia give you give you all the
logistical details on how the the course is organized. I am Leonardo Guerzoni, and I will teach in this course for 3 credits out of the
9 total credits of the main course, and this will, for those of you that are enrolled for the 6-credit course on malware analysis, obviously, will.
Have together a few lectures as well.
I don't think if... I don't know if you have any questions about the logistic organizations, otherwise I will,
Skip the things that you already have, and move straight to the... To the... today's lecture.
Today we'll start looking at the, basics of how to, analyze malware. So, I assume that, Daniele gave you already a, let's say, a basic idea of why we are teaching this, this, this course, and what we expect you to learn by the end of. of the course. As you have seen that from the organization, the course has a... very practical aspect, a very practical flavor. This is the reason why every Friday we meet in the lab.
But to, have a lab lesson that is, let's say, effective, we need to introduce some theoretical, points during the week, such that you arrive on Friday and you will be able to apply those, those.
that knowledge that you acquired in the classroom on on the lab. Okay, on analyzing real malware. So let's get straight to the point. Basic static analysis.
Starting from scratch, why do we want to analyze how malware works? There are several reasons for that.
To, somewhat... Meaningful, reasons here. The first one that

In order to better protect the systems that are.
running within our organization, be it the university or whichever company or public administration you can think about, we would like to be able to monitor and prevent attacks. But preventing attacks means being able to identify the signs, the information that
may possibly alert us about the fact that something bad is happening, okay? So we need to know something about the attacker in order to be able to detect its presence inside our
our networks. How we do that? Well, we study by analyzing, for example, malware. This is not limited, obviously, to malware, but for this talk, for this course.
will deal with the analysis of malware software, analyzing its internals in order to identify which are the recurring attack patterns. Recurring attack patterns can then be later translated in detection rules that we can install on
Automatic devices that will raise an alarm whenever any of these patterns is recognized.
Okay.
So, in the end, what we would like to do is to extract information from our work in order to build rules for monitoring.
By the way, if we are able to understand how malware works, and how it's able to subvert the correct functioning of the system that we... we protect, we should also be able to, let's say, to distill enough information in order to build better systems.
If you know that an attacker is using the lack of proper
proper isolation among the permission, the rights given to some user accounts, okay, let's work on that, let's improve our management of our access control policy in order to protect our systems against this kind of attacks.
The second reason is, being able to, better Manage ongoing incidents.
If you're not good enough at monitoring and preventing attacks, sooner or later, something bad will happen. And when something bad happens.
What we would like to do is to be prepared in order to manage what's happening in the best possible way, in order to restore as fast as possible the, let's say, the safe conditions that were existing before the attack.
So, in order to understand how to correctly respond to an incident, possibly caused by an infection from a malware, we need to understand how that malware works.
okay otherwise we'll not know exactly what the software was able to do
typically when you run a piece of malware that provides

no information about what... about what he's doing, okay? It's a silent piece of software. It's also designed in order to leave as less traces as possible.
So dissecting its internal to understand how it works is important to understand where to look for possible traces left in our systems.
So we would like to understand what the malware actually does in order also to identify all compromised assets.
if I understand that a malware that I'm analyzing has the possibility to replicate itself in other machines, okay, let's take a look at the network log to see if the malware actually opened connections toward other personal computers that are possibly infected.
Otherwise, how could we possibly identify which machines have been infected, and which others have not been infected?
If we know how the malware behaves and what it actually does on our system, we will be able to better contain the expansion of the attack.
By analyzing malware, we may possibly learn that the malware scans the network every night at 3am using a specific protocol, and whenever it finds a machine that has a specific port, TCP port open, it tries to replicate itself.
Okay, let's put a rule on our fiber, internal fibers that block every connections with those specific characteristics. This will,
make it possible for the malware to replicate itself, so it will contain the spread of the infection within our assets. So, as you can see, understanding how the malware works is fundamental in order to correctly manage an infection based on that malware.
There's also a third reason that is not cited here, but we will discuss about that, In two weeks.
that is related to... related to knowing who is the, who is the attacker, okay? The malware is just a piece of software. It has been sent to us by someone that possibly has
I assume bad intentions, is willing to get something from us, maybe steal some data.
or, block some services, for whichever reason, yes, okay? But in order to better prevent its future actions beside the current attack.
We need to learn who he is, not as a person, but how he behaves and which are the motivations that move this person, this individual against us. And this is exactly the topic of the of a branch of security that is called cyber threat intelligence.
We talk about that on Thursday lectures.
And...

cyber threat intelligence is based on the usage of a lot of different information but part of this information comes from the analysis of MALWORK
People that are standing, if you want, there are at least a couple of seats here that are available, huh?
Let's move. Okay, what would we... do we want to extract via our analysis from a malware? Clearly, if we analyze a malware and fully understand how it works, we can write a nice report.
That tells other people how that piece of software works, okay? But in practice, this is only partially useful to the,
Goals that we... that I cited in the previous slide. If you want to write a rule that blocks the,
the replication of a malware inside the network that has been compromised, you don't need a 16-pages document, okay? You just need a rule.
This is exactly part of the things that we want to extract from the malware. In particular, if we
If our first goal is to identify the presence of an infection from this malware inside the network, what we'd like to get from the analysis are a set of signatures. That is, a set of information whose presence in a network
being it in a part of a network, being it on the network logs or in a NOST, that signature should exactly represent the presence of that malware inside that asset.
For example, I may want to identify... I may like to extract from the malware the fact that this malware writes inside the Windows registry a specific key that is represented by a string.
That's great, that is a signature, because if that key, the value of that key, sorry, the name of that key is peculiar, I could possibly write a rule that is checked on each machine deployed in my network, saying if in the Windows registry.
You find this key with that specific name, that is a credible sign of the fact that that machine has been infected by the malware.
If the malware sends messages with a specific HTTP messages with a specific user agent string, okay, let's monitor the network. Whenever I see a message, an HTTP message with that user agent, that is potentially an indication of the fact that in the network that malware is currently spreading.
Okay? All these pieces of information are called signatures, because exactly like the signatures we place on paper.
They typically, with high probability, represent a single individual, in this case, a single, a specific malware.
And signature may take different, shapes.

We have host-based signatures that contain information that can be checked on a single host.
So, as you can see here, the name of files, right, processes with a specific, memory footprint, register keys, or whichever other information I can find on a specific cost. They represent cost signatures.
Whatever is linked to information that can be checked on the network represents network signatures. Again.
Characteristics of configuration of a specific protocol.
rate of message sending also the timing with which messages are sent or received at the host external host that the malware communicates to. All of these are signatures network visit signatures that can be used to correctly identify the presence of a malware.
There's a third family of signatures that is even more important than the previous ones, and we'll see why in the forthcoming lectures. They are behavioral signatures. These signatures don't contain
technical information, but they explain the, the behavior of the malware while it's attacking one of our machines or our network in general. For example, the malware installed itself by writing a mail... Writing a...
file inside the temp directory, and then opens a connection to a, to a social network, where it looks for a specific profile, where it downloads the last 10 messages. Okay?
This represents a behavior of a piece of software. The software does all these actions in this specific order.
So clearly, also behavioral signatures can be used in order to detect the presence of a specific malware. Why are they more important than the previous ones? Because while the attacker may easily change the previous signatures.
For example, we identify that a malware writes a specific file with a given file name in a directory in the Windows file system. Okay, great. We place a rule, and that rule will catch all the instances of that malware.
Then the attacker discovered that we learned this information, and what he does is simply change in the source code the name of the file, recompile everything, and launch again the malware. And our rule is not useful anymore. Okay? This is not true for behavioral signatures. Why?
Well, the point is that also behaviors can be changed, but for the attacker, rewriting the behavior, changing the behavior of its piece of code, of its malware, will typically take a lot more effort, time, and therefore costs, rather than changing any of the previous signatures.
Okay? This is a...

The difference between different kinds of signatures is something that we'll discuss in details when we talk about cyber threat intelligence.
Okay, how do we extract this information from
a malware. In the end, a malware for us is, is an executable file.
That is, written in some form, but that we know that contains binary code. It's written and compiled for some specific.
Architecture, in this course.
32-bit Intel architectures with Windows operating system. We will not look at other architectures and operating systems. So, how do we get all this information from there? This is exactly what we learned by looking at
static and dynamic analysis techniques, okay? Before going to the...
Let's say interesting part on how do you, how do you extract this information, a couple of warnings.
When you... Analyze malware, you are handling a sort of explosive material.
It's something that if it's wrongly analyzed and is executed in a non-controlled environment, can possibly destroy your machine.
It can simply, for example, if you're analyzing a ransomware, it can easily encrypt all the data in your personal computer, okay? So...
This means that Whenever you analyze a piece of malware, you need to...
Know exactly what you're doing.
and follow a set of safety, let's say, instructions in order to avoid that something goes wrong. Now, inside this course, for obvious reasons, we provide you only with samples, with malware samples that are absolutely safe.
They will never delete anything or change anything in a, let's say, adversarial manner on your machine. So, whichever piece of software you analyze that is provided from me and Daniele is perfectly safe.
But if some of you, by the end of the course, decide, okay, but we are looking at fake malware, let's check something real.
Obviously, you can do that, but here, please take a look at these couple of warning slides before doing that, okay?
One important point is,
is related to, network connections, okay? Some... a lot of malware uses network connections in order to, allow its creator to control the machine that has been infected. And we will learn how to identify which are the IP addresses the malware connects to.
Now, the point is that why you're doing it inside this course, okay, no problem, everything is fake, so it's just for fun, okay? With real malware, you need to think that a real malware has been written by a criminal.

Okay? And if you are trying... if you try to connect to the IP address that is contained inside a malware, it's like...
having your,
house robbed by someone, and then you find the home address of your robber, and then you go to his door and knock at his door. It's something you will never do in your real life, okay? So, this is the electronic version of this story.
Avoid to connect live to IP addresses that you find inside the real malware sample, because you don't know what you can expect on the other side.
And in any case.
Do backups.
Okay? Have your data, the data that you think is important.
make a backup of the data, and keep this backup disconnected from your laptop. This is actually a line that should appear in any course of this Master of Science, right? It's valid in general. But in this case, it's even more true.
So, whatever you decide to do, keep a backup of all your data. Basic indications that are always valid. It's a nice idea to perform any analysis of a malware within a virtual machine. So, you install a virtual machine on your on your personal computer. Actually, we will provide you with the... I don't know if Daniele already gave you the link. You can download that virtual machine and perform your analysis inside that virtual machine. As long as the virtual machine is isolated from the host operating system.
You are reasonably safe.
Okay?
I've hardly heard of, malware that is able to surpass the, let's say, the containment measures that are built by a virtualized environment. So, in that case, you're safe.
Remember to disconnect the network of the virtual machine from the internet, unless you know what you're doing and you really need it. So, the best idea is to perform the analysis in complete isolation. By the way, we will learn during this course how to.
Let's say, simulate...
the external network, the internet, for... in order to make the... the malware connect with something and observe what messages are sent by the malware to the... to the... to the external IP addresses without making it actually interact with external addresses.
Okay.
So, how do we analyze a malware? Two main techniques. There are actually more than two, but we'll deal only with two of them during this course.

Static analysis. We examine the malware without running it. So we look at the code.
Okay. Yes. But the code is the the the malware is is compiled. Okay. We will learn how to analyze assembly code. Okay. We'll need to disassemble the malware and try to interpret the assembly code in order to understand what the malware is doing.
This is something that will,
look at in a couple of weeks. Let me start immediately with a recommendation that is valid in general in all this course. We don't need to understand exactly all the minimal details on how the malware works. We need to have a...
Broad idea that is, correct, first of all, and is detailed only to the point
Where it is useful.
Okay? Minimal details are oftentimes completely useless from the point of view of extracting useful signatures. So, here the game will be to use the methodologies that we learned together, with.
always remembering that it's... we need to find the correct trade-off between the effort that we put in the analysis and the result that we get from the effort that we are using, okay? Sometimes.
You will try with a technique and you will quickly discover that after the initial, information that you get, you, you are stuck.
At that point, it's oftentimes not that useful to insist in order to overcome these limitations. It's oftentimes better to try another way that may possibly bring you to the same results with less effort.
Learning how to manage this trade-off is not easy, and it's something that you should try to acquire by the end of this course. Even because Daniele already told you that at the exam you will be asked to analyze a sample that we provide. In a limited time frame.
That is long, it's still 3 hours.
But you have never seen 3 hours pass so quickly, like during the exam. Not analyzing everything to the minimal details in 3 hours is... is not possible. And we do that exactly to try to
Push you in understanding where is the right point to stop the analysis and move to another part.
Okay, let's get back to to our slide. So static analysis, we examine the malware without running it.
And we'll use several tools.
The first tools are automatic. They are the ones that we'll see partially today, and then during the...

Not this Friday, the next one, Friday next week.
And then we'll use a disassembler like IDA in order to look at the code and we learn how to interpret the code.
On the other spectrum, we have dynamic analysis. With dynamic analysis, we run the malware in order to monitor its effect on the system where it is running.
Why do we have these two different approaches? Well...
One may say... one of you may think, okay, but if I'm able to
Disassemble the code, analyze it, and fully understand the code, there's no need to run it.
Okay? Because everything is written there.
In theory, this is true.
Okay, but in order to fully understand how the mower works, you should... you should be able to interpret the code exactly like the CPU of your personal computer does when it's running the code. And this is not exactly easy, okay? A lot of details may be difficult to grasp.
Even because sometimes the state of execution of a piece of software may become very complex.
And when you look... when you perform static analysis, the only thing that you can look at
is the... is the sequence of instructions that are written in the file, okay? But you, for example, you have no idea about the content of the memory and the content of the stack.
You may possibly infer it by the structure of the code, but you typically can do that up to a certain amount of details, and then you get lost into too much information.
And for sure, you don't have an idea about what is the environment where the malware will be executed. Because the environment, you are analyzing it statically, without running it, there is no environment. There is no execution environment.
Okay, so there is also another important point. Oftentimes the malware is obfuscated. That is, the attacker knows that you will try to analyze the malware.
So what it does is, let's cripple the code in order that it's not readable.
And in that case, static analysis is not that effective.
Because most of the code will be unreadable.
So you have the other option. Let's run the malware and let's see what it does while it's running.
Clearly, this solves most of the problems that I cited. You have a full picture of the environment, the execution environment.

You may possibly also look at the content of the memory. You have full visibility of the stack. You can look at what is the status of the CPU while executing the malware. You can see a lot of different information.
And if the malware is obfuscated, well, even in this case, dynamic analysis comes to an end, because
If the static code is obfuscated, in order to execute that code on a CPU, the malware still needs to deobfuscate the code.
Okay? Sooner or later, this will happen in memory. So, if you're patient enough. You look at what's happening, at a certain point, all the original code with no obfuscation will be present somewhere in the memory of your personal computer. Actually, of your virtual machine.
That's great. Why do we need static analysis? Well, we have dynamic analysis that solves all the problem.
Yes, but it has other limitations.
When you dynamically analyze a malware, you're looking just at an execution path of that piece of software, not at all the execution paths.
Think about a complex piece of software that have many branching and conditions that alter its functioning, okay? You may have, for example, a piece of... a set of functions that are executed only if a specific condition is met.
What happens if you start executing the malware to dynamically analyze it and that condition is not met in the testing environment?
Those functions will not be executed, and you will never see them.
So you risk, in this case, to miss part of the behavior of the malware.
This is the reason why, in practice.
all the real-world malware analysts always use a mix of the two techniques. They move from static analysis to dynamic analysis, and vice versa, depending on the point they reached in understanding the behavior of the malware. And whenever they are stuck with a technique, they start with the other one in order to
Make some further steps, and possibly unblock a new evolution on the other with the other approach.
In this course, we, traditionally, I think this is something like the 10th edition of this course, so we are close to that. We divide, following the, the...
the organization of the book that is I will tell you the book that is suggested is old is really old for this topic but it has a still a nice internal organization the analyst techniques are divided in basic and advanced
Now, don't think that basic techniques are useful just to start understanding, but once we learn advanced techniques, we will get rid of them. No, that's not

the case.
Basic means that they are simple to use.
they are quick and the results that you may get from them are somewhat limited
But the fact that they're limited doesn't mean that they are useless.
Okay, actually, the opposite is true. I suggest that you always start your analysis using basic automatic tools, because they will give you a first glimpse on what's inside the malware.
And if there is any information that is potentially interesting, and they will help you guide the usage of the more advanced techniques.
The more advanced techniques obviously will give you more visibility on the internals of the malware, but at price. And the price in this case is time. They are more complex to be used.
And they will require more time to be exercised of the malware. So, it's... we are lucky because we have extremely powerful methodologies that we can use to analyze in-depth malware, but we need to use them carefully because they are very expensive.
And in order to use them wisely, we need to be guided by basic techniques that are cheap and quick.
Okay, this is the concept that you need to keep in mind.
So, for static analysis in particular, we look at two tools that are quite simple.
The first one, strings, as the name suggests, extract textual strings from the code.
Okay? Everything that looks like a string is simply extracted from the code and printed on screen.
Very basic, okay?
Still, you will be surprised how useful this tool is because it's extremely important in order to guide further inspections inside the mower.
The second tool PE well it's a collection of tools PE inspection tools are useful in order to understand how the file the file you are looking at is organized internally.
They will divide the part where the code is contained from part where you have, for example, global variables.
from the part where you have metadata of the executable file. All this information, if correctly analyzed, may provide you with some basic hints about how... which are the characteristics of the malware.
For basic dynamic analysis, we'll use,
A tool inside a virtualized environment that simply monitors the interactions between the malware and the operating system.

Notice that I'm aware
It's a program. It's a program with a funny name. Just a piece of software. And like any piece of software running on a modern computer in order to do anything useful.
It needs to interact with the operating system because modern operating systems are quite good at isolating the software, the application layer to the hardware layer. So if the malware wants to do something with the hardware, for example, send a message to the network, the only thing that he can do is to convince the operating system that he needs to use that.
The the.
The, the, the, the network, the network connection.
So, clearly, given that the malware, like any other program, interacts with the operating system, we would like to understand exactly what it asks to the operating system, and what... which are the answers that the operating system gives back to the malware.
Okay? All this information gives us a lot of,
Info on what the malware is actually doing.
If I see that the malware is interacting with the operating system and in particular with all the part of DS that handles the file system, well, probably that malware is reading or writing something to the disk.
So, that's important. We may look in the code where these functionalities are implemented, and if it is writing a file, what's the content of that file?
Okay, for the advanced techniques, I already spoiled the... The, the first part? Static analysis will be performed by reversing engineering the code using a disassembler. For dynamic analysis, we'll use a debugger. Have you ever used the debugger as a programmer?
Some heads are very slightly moving. And now, okay.
For those of you that never use a debugger, a debugger is a software development tool that allows you to... allows the programmer to run its code in a controlled environment, where it can check step by step
how the execution of a single instruction changed the status of the of the personal computer. So through the debugger you can
execute one instruction at a time and after each instruction you can inspect the status of the CPU of the memory files whatever
Okay? We'll use exactly this tool in order to run the code through the debugger, and see what the debugger is actually doing on the CPU.
If you think that this is,
time consuming.

It is time-consuming, and in fact, this advanced tool, let me repeat, that should be used only on those very tiny parts of the malware, where there is a specific functionality implemented that you think, you suspect is fundamental to understand the behavior of the malware, but that so far.
was not understandable using more basic tools.
Okay.
This is something that we discussed.
Given that we want to extract.
Well, actually, we want to identify the behavior of the malware, which kind of behaviors we can expect from a malware. What the standard malware does on a machine.
If any, if each single malware is a different story, this is true, like, yeah, there's a question.
Sorry?
Would you like?
Yes, let's try to accept some meeting.
Yeah, that's the point.
Let's try to make this exercise for everyone, which kind of behavior you may expect from a model.
Reading, writing, this... Reading, writing on this? Yes, but also non-malware programs to even write that on this.
many files at a time. For example, a ransomware writes a large amount of data on many files. Okay, that's a good point.
Huh? Details.
Because it may possibly perform actions that... whose sole purpose is to make the day of the analyst a bad day, okay? So try to convince the analyst that it's looking at something while the MAU is doing something else. That's a good point. Installing a possible rootkit.
I don't know, open some back doors. Okay, installing something bad on the machine, there are various things that can be installed. Your colleague cited, root kits, back doors, okay, that's perfect. I mean, changing the behavior of the system.
Network traffic, not necessarily strikes, but network traffic, yeah, that is something that's uncommon behavior. So, as you can see, if you look at your colleagues in the... in this room, there are no two people that are identical, okay? But all of us have, we have all hairs.
some more hairs, some... well, that's less hairs, but we all have hairs, we have two eyes. So, the malware is exactly the same. They are different, but they have

common traits, okay? And these common traits is exactly what we would like to categorize and extract by analyzing the malware. So, first.
of... Functionalities that is often implemented by malware is persistence.
Think about a malware that That does, a long and difficult journey in order to infect a machine.
Okay?
By the way, we will not, as probably Daniele already told you, we will not look at part of the
attack where a machine that is protected is actually compromised, and something is, is injected, a piece of code is injected as security. We actually start exactly from that point, okay?
And once a piece of code, a line of code is executed.
The first thing that the malware writer needs to do with its malware is to make sure that the malware survives inside the environment, the compromised environment.
The most obvious thing that can happen to the environment where the malware is running is that at the end of the business day, the personal computer that has been infected will be turned off.
Okay? So the next day, we would like the... when the computer is again turned on, we would like the mower to be there, running. Okay?
So persistence is exactly this behavior. The malware installed itself in the system like any other, similarly to any other software, with the big difference that typically when you install a piece of software, authorized software, that is visible.
it leaves visible traces in the system, the malware will do exactly the same. It will try to install itself in a, let's say, in a non-visible way. It will try to leave as less traces as possible. Luckily, luckily for us.
Installing a malware inside the system typically, more or less, always leaves some traces.
And this is one of the weaknesses of common malware, and the reason why today we also have
cases of what is called file-less malware. Malware that doesn't install itself, doesn't write anything on the disk, it only survives in memory.
Okay? And it guarantees its survivability in different ways. Oftentimes, it has an external mechanism that every time a machine is rebooted.
Recompromise again the same machine.
This is just as a parenthesis.
Second common behavior.

Uniqueness.
Malware code is not written to be elegant, to be correct.
to be something that can be often reused there are some exceptions okay but when you buy a piece of code
a word processor, you download a... but even if you download an open-source piece of software, that piece of software is built with quality in mind, okay? It's written with code that is readable, is well-organized, modularized.
It's written to correctly work in many slightly different environments. It has documentation, and it underwent a lot of testing.
It's built with quality. Malware is typically exactly the opposite. It's not built with quality, okay? It's built by a few people for a specific goal in mind.
but it needs to work for that specific exercise and that's it okay so from this point of view malware code is oftentimes quite fragile
And one thing that malware code typically doesn't like when running is that another instance of the same malware is competing for the same resources.
so one thing that a malware often time does is to check that the current process that is executing is the only one executing the code of that that program
Okay? So it will use a few more or less known techniques in order to perform some checks and make sure that that one is the only copy that is currently running.
Whenever it discovers that there is another already running copy, it will simply quit. Oftentimes it will simply quit without any other notice.
But, parentheses. Notice that there are known cases Of malware that,
Once installed on the compromised system, actually patch the system to avoid compromisation by other malware.
Okay, so they found a machine that was vulnerable, the malware infected that machine, and in order to avoid other malware to compete for the same resources, it actually improved the level of security of that machine.
That is not that intuitive, but... It sometimes happens.
Another common trait of a lot of malware is targeting.
modern malware is not meant to be run on any machine on earth okay this was actually common in the past if I look back at computer security in the 90s beginning of 2000s we were witnessing at the time I mean the common case was the case of a mass malware a piece of software that is left uncontrolled on the internet
And that will try to infect as many machines as possible without looking in details of which is the current victim.

Okay. Modern malware is completely different, because who writes modern malware is interested in,
monetizing, obtaining some economical, typically economical return from the infection. And, even because writing a malware takes some, effort, that is, money, okay?
So, in order to be able to monetize, to get back some return, some economic return from an infection, from an attack, the attacker needs to control the attack, and in particular, needs to control who is the victim of that attack.
So, from this point of view, oftentimes, modern malware contains a few routines whose sole purpose is to check that the malware is actually running on the correct target machine.
We have malware that checks for the current time zone.
For example, there is malware that has been designed only to infect machines in North America. Okay, let's check if we are in the correct time zone for North America.
localization obviously also for language yes I want to affect machines in North America but I'm my origins are from Latin America so let's keep
Everyone that uses Spanish as the main language.
Okay.
And whatever may possibly come up to your mind.
In the most complex cases, the malware may possibly try to stay silent and monitor the usage of the machine in order to collect further information that better characterizes the usage of that specific target in order to be sure that it's actually attacking the correct target.
Obfuscation, we already quickly introduced that, and evasion. This is another very common characteristic of modern malware. Obfuscation, again, means that the code of the malware
shouldn't be readily readable by an analyst so the attacker will use
Various techniques, randomization, it will try to add garbage code, it may possibly use encryption techniques in order to
Make the... the code looks like random stuff.
Okay, this will, hamper the possibility for... will hamper the capacity of the analyst to statically analyze the malware.
Then we also have evasion techniques. Evasion techniques are used against dynamic analysis.
If I'm the malware, and I discover that I'm currently running in a testing environment used for dynamic analysis, I will simply escape, okay? I will quit. I will not show my behavior.

or even worse I will show a behavior that is credible but is not the real one
Do you remember that they've previously told you, look.
Dynamic analysis may look like a very powerful techniques, but always remember that you're looking at a single execution path of the malware.
This is exactly the goal of the evasion techniques. The malware will do its best in order to make sure that the execution path that you are observing through dynamic analysis is the wrong one.
It's not the correct one.
Another common behavior, fingerprinting and beaconing.
What does it mean? Once the malware infects a machine, compromises a machine,
It needs to report back to its creator to say, okay, this is the kind of machine target that I infected and I'm currently controlling.
The, so it takes,
Well, a fingerprint, a short resume of the target that has been compromised and typically tries to contact its creator.
through a beaconing message that is a message that is sent periodically back to a some IP address on the internet. Well, there are also more complex beaconing techniques that is sent on the internet somewhere in some way in order to tell the the its creator, hey, I am here. I I succeeded in
Infecting this machine, and I'm waiting for your command in order to understand what to do.
Out of it.
After beaconing, The subsequent behavior is common and controlled.
While old-fashioned malware was built with a specific functionality in mind.
It was able only to execute this functionality. Modern malware is modular.
It allows the attacker to perform different actions on the target, on the compromised machine.
which action should be executed? Well, when the... after the malware installed itself on the machine, it doesn't know what to do. So, it will contact back with the beaconing phase its creator.
Asking, what should I do next?
Okay? And these commands are sent back
Through a network channel that, as you will see, is,
It's typically hidden, obfuscated, it has several characteristics, but the point is that the malware is waiting for commands. It will receive commands, execute them, and report back about the results to its creator. And this is the communication and control phase.

Notice that inside this phase you can have also some actions on the target. If we have a malware that was designed, was instructed to steal information. These are typically called info stealers. They will collect information on the target on the compromised machine and use a mechanism to send back this information to a place where it will be collected safely by the attacker.
And then, obviously, you may list other kind of actions that are...
less common, they are specific of the... they may change from malware to malware, but they can be, let's say, grouped under the concept of action on objectives. Okay, the malware is installed, it is ready, receives a command, and then it performs an action on the objective. This action could be
still information.
It could be, send an email using the local email address. It could be encrypt all the data with this password. It can be whatever may possibly come up to your mind.
Okay, don't get caught in details, this is what we already discussed enough.
Okay, let me move quickly to the first techniques that we can... we could use. In order to analyze a malware, let's assume that we know nothing about analyzing malware, okay?
Which is the first thing that may possibly come up to your mind to analyze the malware? Well, we have antivirus software, let's use it.
Okay? And that perfectly makes sense. There are variants of antivirus software that are actually used also by people working in the area. One example is VirusTotal, but there are other similar examples on the web.
Oh.
What VirusTotal does is... you may upload a file like something that you suspect is a malware
It will analyze the malware with a lot of different tools and antiviruses, and then at the end, it will return you with a score, saying, yes, this is probably a malware, no, this is not a malware, okay?
Umm.
so the functionality may look very basic actually this instrument is way more complex
the, the free tire only allows you to send a file, and, it will report back is that file is, how much that file is suspicious, okay? It will actually... you, you can try it. It will give... don't send the private data to Uber Estotal, please.
but take a copy of, I don't know, the Bible or whatever, send it to VirusTotal, and you will see that it will return a list of antiviruses' names, and for each of them, it will provide you with a

score that says okay for this antivirus your file is malware or it's not a malware. And.
in the backend, what Verisotal is actually doing is that it will perform what we are learning here, okay? It will analyze with a lot of automatic tools the malware, it will extract a lot of information, and will place that information in a global database.
That... whose content are then sold to companies that are interested, obviously, in studying how malware typically works.
But the point here why I told you don't send sensible or private data because your data will end up in that database. Okay? And you typically don't want that to happen.
Notice that... This is, by the way, a problem for us, because let's assume that you receive an email with an attachment, okay? And you... you would like to open that attachment, but you are not that sure... I mean, it may possibly be a virus, okay? So let's check it with the... with the virus total. There are two options. You were wrong, you were just paranoic.
Welcome to our family. But luckily, the Aristotle tells you, no, don't worry, you can open it safely, okay?
The other case is, hey, you were right, this is a malware, hmm?
In that case, there's a problem, because if the malware was not sent to anyone else before you.
the attacker will know that you analyze it on VirusTotal. Why? Because when you analyze a file on VirusTotal, it will retain the file, and will,
let's say, we will calculate a signature, a fingerprint of that file, such that if someone else sends again back that same file, Veristotal will not re-execute the analysis for a second time, because it's a waste of time and resources. It will simply reply back with the original analysis results.
So what the attack and this is provided with the search functionality.
Okay, so what the attacker will do is simply periodically check with the search functionality if his own malware has been analyzed. As long as no analysis is available, he knows that you probably are not aware of the fact that he tries to infect you.
But if it finds in the database.
The malware that he sent you, he knows that you already discovered its actions and he will move to a different kind of attack.
Okay, so be careful with the information that is shared explicitly or implicitly via Verus Total.
By the way, you will see that beside Verusol, there are other, platforms with a

free tire.
That actually provides you with,
In-depth report of the, of the malware content, okay?
There are platforms that give you
Hundreds of pages of report, of technical report.
And it's up to you to decide what to do about the content of those reports.
They're not easy to read, let's say.
Okay, I cited the fact that Verstotal compute a signature of the file that is uploaded. The signature is typically calculated using an ash function.
the hash function is, computed on the content of the file that you upload, and it represents, the de facto standard way of identifying in a unique way, a specific sample. Notice the difference between the concept of sample
and malware. We may have several samples for the same malware.
what does it mean let's assume that criminal has a code base
For a malware. It has the source code.
it compiles the source code and obtains an executable file. That executable file is a sample, okay?
it sends the sample out, and after a while, that sample ends up on VirusTotal, because someone discovered it, okay? Once it's there, it's known. It will be in any antivirus database, so it's... that sample is completely useless. It will not infect anyone in the world.
So what the attacker could possibly do, quite cheap solution to this problem, is that it will get back to the code base, it will change a completely useless detail, just moving a parenthesis, adding a line code, a line to the code that does nothing, and it will recompile the code.
the executable file that it obtains is different from the original one. And in particular, if you apply an hash function to the two executable files, you will obtain two signatures that are completely different, because this is the typical characteristic of hash functions. Even if
The two executable files.
come from the same codebase, and they have very slight differences. The signatures will be completely different.
Okay.
So this means for the attacker that he will be able to send again this malware in the wild and possibly antiviruses will not be able to identify them, to identify that. Why? Because if they use signatures in order to identify known viruses, well, that virus is a variant that is unknown. What you're looking at are two samples, two different samples.

for the same malware. The malware is characterized by the behaviors and the technical details. The sample is represented by the binary code that implements the functionalities of the malware.
Okay.
Keep this, this difference in mind.
Okay, let me skip things that are not that useful. Okay, strings. I told you that a basic tool for static analysis is represented by a program called Strings, that what it does is simply to extract strings from a mower. Why is that made possible that
How could possibly be useful? Well, if you execute in a, that on, on some file that you
You deem suspicious. You may end up with something like that, okay? So, there are strings here.
That are with high probability, at least to me, they look like random strings.
They are...
Three characters like this one, T dollar at sign, or this one, E dash at sign.
they probably don't mean anything okay they are just noise
why they appear here well because this is due to how the strings program extract string data from the executable file it simply looks at the chunks of binary data
looking for the null character. Every time it finds a null character, it takes all the previous characters that are part of the ASCII table and print them here.
Clearly you can have some false positives, stuff that are bytes that are not actually part of a string.
But still, they look at, like, strings to the, to the program.
Hmm?
Inside this noise, there are other things that instead they absolutely look like strings, and they may have different meanings. This one is clearly an IP address.
Here we have... GDI 32.dll. This is the name of a dynamic link library. It's a Windows library.
Then we have get layout, set layout. These are two known names of methods from this library.
Then we have a long message. Mail system DLL is invalid. Send mail failed to send message.
Okay, this looks like a debug message intended for human beings.
So, all this data may provide you with some basic information. Which information can you extract from here? Well.

First of all, it seems like this program includes some network functionalities. Otherwise, why including an IP address inside the program?
So, it may possibly contain some functions that open connections toward the internet, and in particular, we may be interested in checking who is behind this IP address.
Put this IP address on Google and check if it is a known source of suspicious messages, for example.
One more thing that we know is that this software has a user interface, okay? Why? Because GDI32DLL is a known Windows library used for building user interfaces, and in particular, getLayout and setLayout are two functionalities that are needed in order to manage correctly
Windows on screens.
Probably if you run this software, that was not a malware by the way, it will print something, it will have a user interface on the screen.
And so, as you can see, by looking just at the strings that are extracted, removing all... everything that looks like,
obvious noise, you end up finding interesting information. This information can be used to, let's say, describe in detail the behavior of the malware. Obviously not, because that is not enough. But this is very useful to understand
what to look for inside the code of the malware. Oh, I found an IP address. Where could I possibly find the code that performs the connection to the internet in order to understand which data is sent?
well let me check where this string appear inside the code and from there let me look back at the code that is actually reading and using that string and you will probably end up in the part of the code that is the one that you're interested in analyzing more in detail notice that this approach is fundamental because as you will see when you
Disassemble even a very simple program.
you will end up with thousands of lines of assembly code, and you don't have time and the need to understand thousands of lines of assembly code. So you need some way, a mechanism, a methodology that allows you to pinpoint exactly where to look for interesting things. And one is, this is a basic, but, I mean, efficient approach.
Obfuscation through packing. I will not spend that much time on that because you will have a week dedicated from Daniele on packing.
Packing is a simple obfuscation, well, not that simple, it's a widely used obfuscation mechanism, and it was actually designed not for malware, for different purposes.

Let's get back to the 90s. Basic computers with very little memory available. Okay? Very small hard drives, 40 megabytes, stuff like that.
They filled up in a matter of a few days. Okay, so someone said, okay, but if I have a program that needs to be stored inside a smaller drive, I would like that program to take as less space as possible.
Okay, so how could I possibly make that happen? Let's compress the code of the executable code.
Okay, but if you compress the code, and you store it compressed on disk, if you want to execute it, you need first to decompress it, and doing it manually is cumbersome, it's not that nice, okay? So they invented packing mechanisms.
A packing mechanism is a tool that allows you to compress and decompress the code on the fly. So, the APACA takes the original executable.
Compress it, place it in a portion of the new executable file, and adds a wrapper program whose sole purpose is to take the compressed part, once the program is executed, take the compressed part, decompress it in memory, and execute it.
And here you have the advantage because while stored on disk, it will take less space. While executed in memory, it will be automatically decompressed.
Okay.
nice now back to our current times this is used by attackers in order to hide the original code why? because once you compress with standard techniques executable code you end up with something that looks like random characters okay at least at the first glance
Your goal is to understand, first of all, when code is obfuscated through packing.
And then with Daniele, you will learn how to identify the packer whenever it is possible. There are packers that are well known because they are freely available tools.
But also when the packer is built on purpose by the attacker. In that case, you need to learn how to unpack the code. So how to rebuild the original executable starting from the packet version.
By the way, in the virtual machine, you have this PEID program that allows you to analyze automatically the
The file and provides you with a guess on the packet that has been used.
Okay, one further... point that is relatively long before ending today's lecture at 4.
Done.
Do you know what happens to an executable file when you execute it?

Because it's nice to say, yes, executable file is something that you can execute. Okay, but the file is on the disk.
The execution happens in the CPU using the memory.
What's in the middle?
There's the kernel, but the kernel does a few things, okay, in order for that to happen. And in particular, one point is that there is a specific component of the operating system that is called the loader.
Whose main purpose is to read the content of the executable file, extract from the executable file all the various elements that makes the program, and place them correctly in a memory that is going to be allocated for the execution of the program.
Once the memory has been allocated, it will populate all the necessary data structures in the operating system, in the kernel, in order to support the runtime of the execution at runtime of that program.
Okay? So a lot of things actually happen. But what we're interested here, what we're interested in here is understanding when the loader
looks inside the code, the program written on disk. How can it distinguish
Where the instructions are written with respect to where is data related to the user interface, where are the metadata, where is, for example, audio or video that is included in the application that is loading, how is organized internally an executable file written on disk.
Well, clearly, operating system follows standards for that.
And there are different standards for different operating systems. But all standards more or less do exactly the same thing. They define the structure of the file, dividing metadata.
from the content of the file and then they organize different content of the file in different parts that are that can be let's say correctly separated okay because
In the... inside the file, you will have different kind of data that need to be treated differently.
In the Windows world, this organization is based on the so-called Portable Execution File Format, or more briefly, PE file format.
That exactly define this stuff.
The file format, it actually comes from other older file formats.
But it's used still today after
35 years of evolution of the modern Windows platform.
And it's used to encode both executable files, programs, and dynamic link libraries. This makes sense, because in any case, we have code in both, okay?

This is the basic organization of a PE file. You see that it has a couple of DOS header and sub, so a couple of blocks at the top.
These are used for, compatibility reasons.
with MS the MS-DOS operating system. Now you may wonder is there someone that is using still the MS-DOS operating system but for running vintage games? Probably not. But it is there since 30 years and I assume that it will remain there for the next 30 years. Okay.
In fact, it's the part that is not interesting for us.
Then we have...
an header that is divided into parts, the portable execution header, and the so-called section table. And we look in details their content, we understand their content details.
And then we have a list of sections. Each section contains a different kind of data, so you may have a section dedicated to, executable code, a section dedicated to.
Data, for example, global variables, variables, constants, and stuff like that.
You may have a section that is dedicated to multimedia data if the program needs that. If the program has a graphic user interface, there will be a section dedicated to elements of the graphical user interface and so on and so forth.
You may have, I don't remember which is the maximum number of section is very large. In a common file, you may possibly end up looking at from.
3, 2, 6, 7, 8 sections, no more than that.
the section table.
describes the various sections, how they're organized, and in particular, as you will see, the section table contains some information about the offset in the file.
Notice that we are looking at the file here. The offset from the beginning of a file where a section begins, such that sections can be clearly divided.
Even because when the loader will load the content of section, each section will end up in a different part of the memory footprint of the program.
Depending on its content.
Each section has a name that is purely, I mean, the name doesn't have any operative meaning. It's just used for humans.
Common names are in the Microsoft Word are .text. The .text section is the one that typically contains code, executable code.
We may have sections named the dot data or dot our data or dot BSS and they contain global variables, constants, textual data, all this stuff.
They don't contain... they can't contain executable code.
Then we have the research section that contains graphical element.

We may have a debug information section that is useful if the software needs to be debugged. So for example, this will contain.
The name of variables from the original source code, it will contain relationship between each assembly code and the original.
language code that produces that assembly line, all the information that is useful for debugging a file.
Notice that the names are irrelevant for the execution, okay? And this has a consequence for us. They can be changed. A programmer may choose different names for the section of its own executable, and oftentimes in malware, these sections are renamed.
Let me add here, we'll discuss, we'll see that also later. Each section also have a set of,
characteristics
that are used by the operating system in order to understand how to treat the content of that section. For example, one fundamental characteristics are.
execution rights.
The text section has the execution flag on.
It means for the operating system that data contained there can be interpreted like,
instruction, assembly instructions that can be executed in the CPU.
All the other sections typically have that flag turned off.
They can use it for something else, but they cannot be executed.
And from the same point of view, why in the data and our data sections you have the writable flag turned on because during the execution the program may possibly write data in those sections once they are transferred in memory.
They need contained data, and data can change over the execution, okay? The same flag, writable flag, is off for the text section. It's not possible to write data or information in the text section why? for security reasons otherwise the program may possibly rewrite itself
And change the set of execution that represent its own code.
And in fact, this is exactly what typically attacker try to do using vulnerabilities in order to write their own code in order to subvert the execution of a running program.
Okay, let's take a look a little more in details about the PA structure. So if you correctly recall at the beginning, we say that the first two parts of the PA file structure is made up by two DOS, the DOS header and the DOS stub.
They, nowadays, they are completely standardized.
The first two bytes,

Sorry, yes, the first two bytes of the...
DOS header have a fixed value that is this, X string 5A4D.
And if you look at it as a text, it's represented by the letter MZ. And this is the so-called magic signature of PE files.
every file that in Windows start with these two characters may be interpreted like a PE file, okay? Then obviously it needs to follow all the rest of the standard. But if you try to execute something that starts with MZ, the loader will try to load it in memory.
Then there is a DOS header whose content is standard, and the only thing that is non-standard are the last four bytes that is a pointer to the beginning of the PE header.
So, at the end of the DOS header, there are 4 bytes that represent the,
the offset from the beginning of the file, where the first byte of the PE header actually starts.
Why this is needed? Because the length of the DOS stub
that immediately follows is non-standard. It may be small, large, whatever.
Why? Because here, the idea of the DOS tab is that it should contain
code that is needed to implement the Windows program.
in a reduced version that can be executed in the MS-DOS operating system.
Okay? If you look at the... if you, look at whichever program, Windows program, you may come up to your... on your PC,
with the next viewer, you will see that here, this part is, again, standardized, but there is a... there are a few strange characters, but there is a string that says this program must be run in Microsoft Windows.
and so the only thing that this the program does is executed in MS-DOS is printing this this string okay so completely wasted space that is there since several decades and we remain there
Okay.
Yeah, this is what you see in, in RX Viewer, okay? So, here we have the 4D5A, the 2 bytes, the MZ, signature.
is here. Then we have here the last four bytes are the pointer, the offset to the beginning of the P header that is here is the beginning of the P header. And then we have the DOS stub that in this case only prints the program must be run under Win32.
The theater... Starts again with 4 bytes that represent another signature.
That is PA, followed by two non-printable characters.
Okay.
They are represented by 5, 0, 4, 5, and then 2 null characters.

Mmm.
The... PE header is quite more complex.
So, we have the 4 bytes with the signature.
Then we have this part here that is relatively small, that's called the COF file header. It's called COF file header because it comes from a previous ancient standard that is the COF standard that Microsoft took and extended to build the P header.
then we have noticed that this part is not optional, okay? Every PE5 needs to have this part.
And then we have the optional headers that are large, 224 bytes, that are...
Actually, they are potentially optional, but they are present in any PFI that I've ever seen in my life, okay? So, it's optional just in theory.
Inside the COF header, you see a few information that are useful.
For example, here we have 2 bytes.
that can be used... that are used to define the number of sections defined in the file. Okay, remember the sections are the one at the bottom of the organization. Each section contains different kind of data. Here, we have the number of sections defined in this file.
Then we have a set of flags, characteristics, that are used for, I mean, indicating various aspects, providing various information on this file. There is one that is interesting that says to the operating system if this file is an executable file or a dynamic link library.
Inside the optional headers, we have stuff that I think is a little more interesting.
Okay.
Yes.
Okay, think for a moment about... this point.
The loader is interpreting this file with this structure.
It needs to, first of all, identify the section that contains the,
The code to be executed, copy that section in memory somewhere, and then it will need to identify, where the... which is the address.
in memory of the first, instruction that need to be executed for that program, okay? All this information is written here. We have address of entry point. This is the address in memory of the first, the, the, the first instruction to be executed.
Okay, notice that this is not an absolute memory address. This is a relative virtual address that is is calculated as an offset with respect to a
an address that is contained here, the image base. The image base is an absolute memory address that represents the starting point from which the

sections must be loaded.
Usually, in the Windows world, this number is 400,000.
Yes, X value.
But can possibly be changed.
Starting from that point, The,
The loader knows where the first instruction to be executed is by adding this relative virtual address to that image base.
One more important point.
Modern versions of Windows
don't adhere to this standard, okay? So here we have the address from which the section should be loaded in memory. What the operating system actually does is to relocate the memory location of sections for security reasons, and this memory... this location is typically randomized.
Okay? So it will randomize the starting address in memory of the image base.
But the important point is that, given that all addresses are expressed like relative virtual addresses to the image base, it can, with a few mathematical sums, it can easily move all the code in the correct position.
These two other information, section alignment and file alignment, are used in order to declare different alignments for the file on disk.
and file in memory.
you should know that when you write a file on disk, data is typically written in chunk of a given minimum amount, such that all data has a specific alignment, okay? In particular, typically, this alignment is oftentimes
512 bytes.
And this is an information that the loader needs to take into account to understand exactly where a section on disk ends.
When this data is transferred in memory, also memory has an alignment problem, but typically, given that memory and file on disk are based on different technologies, the alignment is different. So we also have
Four bytes dedicated to section alignment in memory and in 32-bit Windows operating system, this is typically four bytes.
I don't remember for 64 bits, but 32 bits is still 24 bytes.
Then we have the sides, of the beam image.
In memory, full size. This is important because it will be used by the loader in order to allocate the correct amount of memory.
And here we have sizable headers that is used to understand where the first section in PA file actually starts.
After the optional headers, we have the section table, and in the section table,

we have whose sizes can change from 5 to 5. We have a block of 40 bytes for each section, okay?
The block contains the name of the section, 8, 8 bytes.
Then we have the sides of that section on disk.
And the size of that section in memory.
Obviously, the two sizes can potentially be different.
Why?
Well, that depends on the section. If I have a section containing code.
That code will be on disk, and probably needs to be transferred as is in memory. So probably, for a text section, the size of the...
The size of it in memory on disk will be very similar. It may differ from a few bytes due to different alignments.
But if I look at the section that will contain data, probably this data will materialize only when the program is running. The data is not existent when the program is on disk. So it will...
take a few bytes on disk, but it will take a lot more memory when the file is actually loaded for execution. Okay? So some sections have widely different sizes here, other sections have similar sizes.
Then we have the relative virtual address of the section and the offset of that same section in the file. The characteristics of the section, if it contains writable data, executable data, are reported in this bitmask.
And then we have the dissections.
Notice that in the end.
We end up with something like this. On this, we have that all the sections are usually
placed, beside a small offset due to alignment, they are placed one after the other in order to make the file take as less space as possible on disk, okay?
Then they are loaded
In memory and in memory can be placed in different, in different positions that depends on the operating system. The operating system typically allocates, Memory space that is defined by the virtual memory system.
And inside this memory space, it will reserve parts to different sections. Some sections will place it at the beginning, others at the end. It depends on the conventions used by different things.
Okay, I think it's 1 minute past 4.
It's enough. I will, when we see each other in the lab in two weeks, no, not this Friday. Friday of next week, I will need to spend five minutes to explain, libraries, okay? Let's, don't forget about that.